

Integración de Firebase Firestore en un proyecto Ionic 8 + Angular (Standalone)

Guía paso a paso para implementar un CRUD de usuarios conectado a Cloud Firestore
en modo desarrollo (localhost / `ionic serve`)

Proyecto de ejemplo: `electivai202603uam`

Guía paso a paso

1. Crear el proyecto y la base de datos en la consola de Firebase

1. Entrar a `https://console.firebase.google.com/` y crear (o abrir) el proyecto.
2. Ir a **Configuración del proyecto** (ícono de engranaje) → pestaña **General** → sección "Tus apps" → agregar una app **Web** (ícono `</>`).
3. Copiar el objeto `firebaseConfig` que Firebase genera (contiene `apiKey`, `authDomain`, `projectId`, `storageBucket`, `messagingSenderId`, `appId`).
4. En el menú lateral ir a **Compilación** → **Firestore Database** → clic en **Crear base de datos**.
5. Seleccionar **Modo de prueba** ("Test mode"). Esto genera reglas de acceso abierto con vencimiento a 30 días, útiles solo para desarrollo:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.time < timestamp.date(2026, 10, 4);
    }
  }
}
```

6. Elegir la ubicación del servidor sugerida por defecto y confirmar.

2. Instalar las librerías necesarias

Desde la raíz del proyecto Ionic, ejecutar en la terminal:

```
npm install firebase@^11.8.0 --save
npm install @angular/fire@latest --save
npm dedupe
```

Librería	Propósito
<code>firebase</code>	SDK oficial de Firebase (cliente JavaScript/TypeScript)
<code>@angular/fire</code>	Wrapper de Angular sobre el SDK de Firebase; expone APIs inyectables (<code>Firestore</code> , <code>provideFirestore</code> , etc.) y observables compatibles con RxJS

Verificar con `npm ls firebase @angular/fire` que ambas dependencias resuelven a una única versión de `firebase` (sin duplicados anidados), ya que `@angular/fire` exige un rango específico (revisar su `package.json`, campo `dependencies.firebase`).

3. Guardar la configuración de Firebase en los `environments` de Angular

Archivo: `src/environments/environment.ts`

```
export const environment = {
  production: false,
  firebaseConfig: {
    apiKey: 'TU_API_KEY',
    authDomain: 'TU_PROYECTO0.firebaseio.com',
    projectId: 'TU_PROYECTO0',
    storageBucket: 'TU_PROYECTO0.firebaseio.com',
    messagingSenderId: 'TU_SENDER_ID',
    appId: 'TU_APP_ID',
  },
};
```

Archivo: `src/environments/environment.prod.ts` (mismo contenido, con `production: true`):

```
export const environment = {
  production: true,
  firebaseConfig: {
    apiKey: 'TU_API_KEY',
    authDomain: 'TU_PROYECTO0.firebaseio.com',
    projectId: 'TU_PROYECTO0',
    storageBucket: 'TU_PROYECTO0.firebaseio.com',
    messagingSenderId: 'TU_SENDER_ID',
    appId: 'TU_APP_ID',
  },
};
```

4. Registrar los providers de Firebase/Firestore en el bootstrap de la app

Archivo: `src/main.ts`

Agregar las importaciones de `@angular/fire/app` y `@angular/fire/firestore`, y añadir los dos providers nuevos dentro del arreglo `providers` de `bootstrapApplication`:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular/standalone';
import { provideFirebaseApp, initializeApp } from '@angular/fire/app';
import { provideFirestore, getFirestore } from '@angular/fire/firestore';

import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';
import { environment } from './environments/environment';

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideFirebaseApp(),
    initializeApp(environment.firebaseConfig),
    provideFirestore(),
    getFirestore(),
  ],
})
```

```

    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideFirebaseApp(() => initializeApp(environment.firebaseConfig)),
    provideFirestore(() => getFirestore()),
  ],
});

```

`provideFirebaseApp` inicializa la conexión con el proyecto de Firebase usando las credenciales del `environment`. `provideFirestore` habilita el módulo de base de datos Firestore para toda la aplicación standalone.

5. Definir/ajustar el modelo de datos

Archivo: `src/app/models/interfaz.ts`

Firestore genera IDs de documento como cadenas de texto, no como números autoincrementales. Por eso el campo `id` debe ser `string`:

```

export interface PersonaI {
  id: string;
  nombre: string;
  edad: number;
}

```

6. Crear el directorio y el servicio de acceso a Firestore

Crear el directorio de servicios (si el proyecto no lo tiene aún): `src/app/services/`

Dentro de ese directorio, crear el archivo: `src/app/services/usuarios.service.ts`

```

import { Injectable, inject, signal, EnvironmentInjector, runInInjectionContext } from '@angular/core';
import {
  Firestore,
  collection,
  collectionData,
  addDoc,
  updateDoc,
  deleteDoc,
  doc,
} from '@angular/fire/firestore';
import { PersonaI } from 'src/app/models/interfaz';

@Injectable({
  providedIn: 'root',
})
export class UsuariosService {
  private firestore = inject(Firestore);
  private injector = inject(EnvironmentInjector);

```

```

private usuariosRef = collection(this.firestore, 'usuarios');

public misUsuarios = signal<PersonaI[]>([]);

constructor() {
  // collectionData emite fuera del contexto de inyección; se restaura con runInInje
  runInInjectionContext(this.injector, () => {
    collectionData(this.usuariosRef, { idField: 'id' }).subscribe(
      (usuarios) => this.misUsuarios.set(usuarios as PersonaI[]),
    );
  });
}

agregarUsuario(usuario: Omit<PersonaI, 'id'>) {
  return addDoc(this.usuariosRef, usuario);
}

editarUsuario(id: string, usuario: Partial<Omit<PersonaI, 'id'>>) {
  const usuarioDoc = doc(this.firestore, `usuarios/${'$'}{id}`);
  return updateDoc(usuarioDoc, usuario);
}

eliminarUsuario(id: string) {
  const usuarioDoc = doc(this.firestore, `usuarios/${'$'}{id}`);
  return deleteDoc(usuarioDoc);
}
}

```

Función	Qué hace
<code>collection(firestore, 'usuarios')</code>	Crea la referencia a la colección <code>usuarios</code> dentro de Firestore.
<code>collectionData(..., { idField: 'id' })</code>	Escucha en tiempo real los cambios de la colección y expone el ID del documento como propiedad <code>id</code> .
<code>addDoc</code>	Crea un documento nuevo con ID autogenerado (usado por "agregar usuario").
<code>updateDoc</code>	Actualiza campos puntuales de un documento existente (usado por "editar usuario").
<code>deleteDoc</code>	Elimina un documento (usado por "eliminar usuario").
<code>runInInjectionContext</code>	Evita el warning de Angular por llamar APIs de Firebase fuera del contexto de inyección dentro de un callback asíncrono.

7. Conectar la página/componente al servicio

Archivo: `src/app/pages/usuarios/usuarios.page.ts`

7.1. Importar el servicio y quitar el import de `signal` si ya no se usa directamente:

```
import { Component, inject, OnInit } from '@angular/core';
...
import { PersonaI } from 'src/app/models/interfaz';
import { UsuariosService } from 'src/app/services/usuarios.service';
```

7.2. Inyectar el servicio y enlazar el signal de usuarios (reemplaza el arreglo local "quemado"):

```
export class UsuariosPage implements OnInit {
  private alertController = inject(AlertController);
  private usuariosService = inject(UsuariosService);

  public usuarioSeleccionado: PersonaI | null = null;

  public misUsuarios = this.usuariosService.misUsuarios;
  ...
}
```

7.3. En el método de **agregar**, reemplazar la creación local por la llamada al servicio (dentro del **handler** del botón "Guardar"):

```
handler: (data) => {
  this.usuariosService.agregarUsuario({
    nombre: data.nombre,
    edad: parseInt(data.edad, 10),
  });
},
```

7.4. En el método de **eliminar**:

```
public eliminarUsuario(usuario: PersonaI): void {
  this.usuariosService.eliminarUsuario(usuario.id);
}
```

7.5. En el método de **editar**, dentro del **handler** del botón "Actualizar":

```
handler: (data) => {
  if (data.nombre && data.edad) {
    this.usuariosService.editarUsuario(usuario.id, {
      nombre: data.nombre,
      edad: Number(data.edad),
    });
  }
},
```

8. Enlazar la plantilla HTML con los datos del servicio

Archivo: `src/app/pages/usuarios/usuarios.page.html`

La plantilla consume directamente el signal `misUsuarios()` y llama a los métodos del componente (que a su vez delegan en el servicio):

```
@for (usuario of misUsuarios(); track usuario.id; let index = $index) {  
  <ion-item-sliding>  
    <ion-item (click)="validarUsuario(usuario)">  
      <ion-label>{{ usuario.nombre }}</ion-label>  
    </ion-item>  
    <ion-item-options side="end">  
      <ion-item-option color="warning" (click)="editarUsuario(usuario)">  
        <ion-icon name="create-outline"></ion-icon>  
      </ion-item-option>  
      <ion-item-option color="danger" (click)="eliminarUsuario(usuario)">  
        <ion-icon name="trash-outline"></ion-icon>  
      </ion-item-option>  
    </ion-item-options>  
  </ion-item-sliding>  
} @empty {  
  <ion-item>  
    <ion-label>No hay usuarios disponibles.</ion-label>  
  </ion-item>  
}
```

9. Ejecutar y validar en local

```
ionic serve
```

1. Abrir la página de usuarios en el navegador.
2. Agregar un usuario nuevo con el botón `+`.
3. Editar y eliminar el usuario creado para validar los tres métodos del CRUD.
4. Confirmar en la consola de Firebase (**Firestore Database** → **pestaña Datos**) que la colección `usuarios` refleja en tiempo real cada operación realizada desde la app.

10. Checklist para replicar en otro proyecto

1. Crear proyecto y app web en la consola de Firebase; copiar `firebaseConfig`.
2. Crear la base de datos Firestore en modo de prueba.
3. Instalar dependencias: `firebase` (versión compatible) y `@angular/fire`; verificar con `npm ls` que no haya duplicados.
4. Pegar `firebaseConfig` en `environment.ts` / `environment.prod.ts`.
5. Agregar `provideFirebaseApp` y `provideFirestore` en `main.ts` (o en `app.config.ts` si el proyecto usa ese patrón en vez de standalone puro).
6. Ajustar los modelos que usan `id` para que sean de tipo `string`.

7. Crear el directorio `src/app/services/` (si no existe) y dentro un servicio `Injectable` con `collection`, `collectionData`, `addDoc`, `updateDoc`, `deleteDoc`, `doc` de `@angular/fire/firestore`.
8. Consumir el servicio desde el componente/página, enlazando su signal a la plantilla.
9. Probar con `ionic serve` y validar los datos en la consola de Firebase.